



Міністерство освіти і науки України
Національний технічний університет України
“Київський політехнічний інститут імені Ігоря Сікорського”
Факультет інформатики та обчислювальної техніки Кафедра
інформаційних систем та технологій

Лабораторна робота №8
із дисципліни «**Технології розроблення програмного забезпечення**»
Патерни проектування.

Виконав:

студент групи ІА–34

Оборський Дмитро Вікторович

Київ 2025

Тема: Патерни проектування.

Мета: Вивчити структуру шаблонів «Composite», «Flyweight» (Пристосуванець), «Interpreter», «Visitor» та навчитися застосовувати їх в реалізації програмної системи.

14. Архіватор (strategy, adapter, factory method, facade, visitor, p2p)

Архіватор повинен являти собою візуальний додаток з можливістю створення і редагування архівів різного типу (.tar.gz, .zip, .rar, .ace) – додавання/ видалення файлів / папок, редагування метаданих (по можливості), перевірка checksum архівів, тестування архівів на наявність пошкоджень, розбиття архівів на частини.

Теоретичні відомості

1.1.1. Шаблон «Composite»

Призначення: Шаблон використовується для складання об'єктів в деревоподібну структуру для подання ієрархій типу «частина цілого». Даний шаблон дозволяє уніфіковано обробляти як поодинокі об'єкти, так і об'єкти з вкладеністю [6].

Простим прикладом може служити складання компонентів всередині звичайної форми. Форма може містити дочірні елементи (поля для введення тексту, цифр, написи, малюнки тощо); дочірні елементи можуть в свою чергу містити інші елементи. Наприклад, при виконанні операції розтягування форми необхідно, щоб вся ієрархія розтягнулася відповідним чином. В такому випадку

форма розглядається як композитний об'єкт і операція розтягування застосовується до всіх дочірніх елементів рекурсивно.

Даний шаблон зручно використовувати при необхідності подання та обробки ієрархій об'єктів. Крім того, патерн «Composite» (Компонувальник) краще використовувати, коли ви представляєте структуру даних у вигляді дерева.

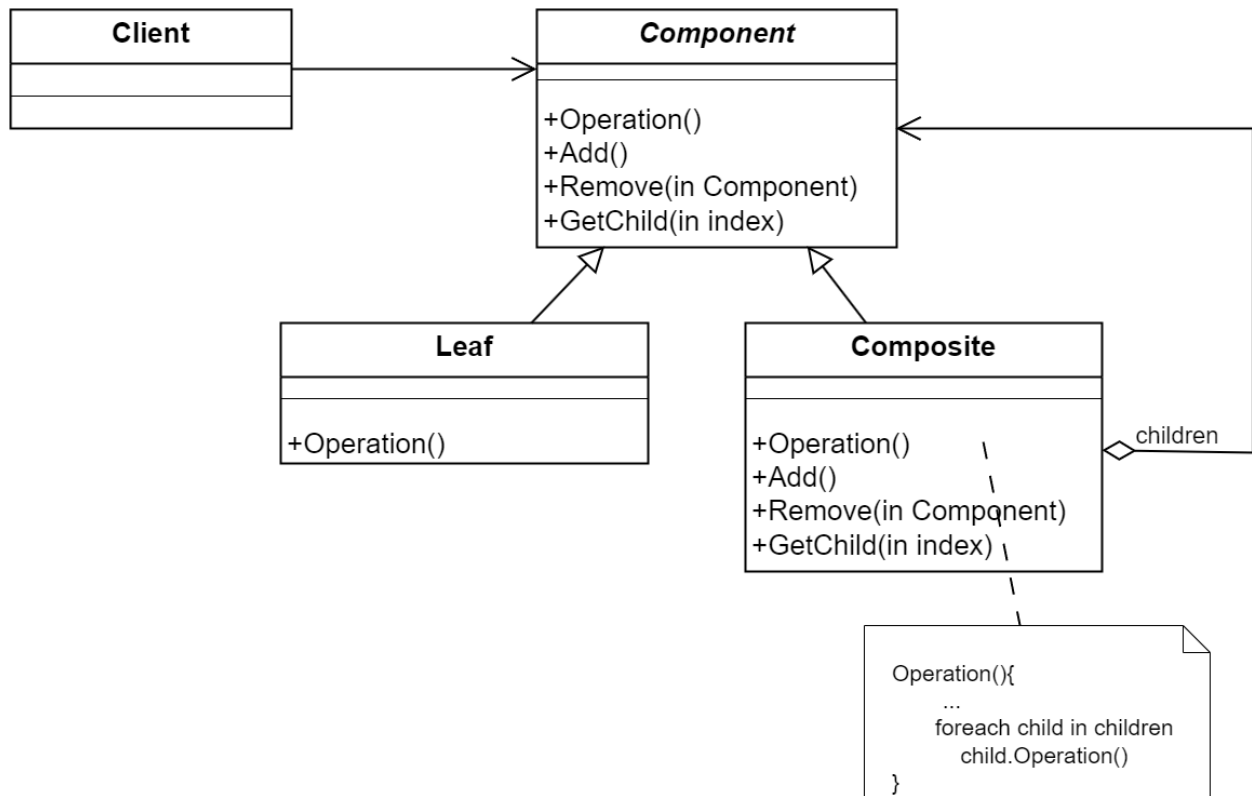


Рисунок 8.1. Структура патерна «Компонувальник»

Проблема: Ви розробляєте систему керування проєктами. Кожен проєкт складається із наборів функцій, кожна функція з userstory, а кожна userstory в свою чергу із задач по її реалізації. Ви реалізуєте функціонал відображення оціночної вартості робіт по кожній із функцій, а також відображення чи всі userstory були оцінені. Це потрібно бізнес-аналітики, щоб розуміти, що всі userstory розробниками були розглянуті і оцінені, а також обговорити необхідність реалізації того чи іншого функціоналу на основі попередньої оцінки.

Рішення: Крашим підходом в даній ситуації буде використання патерну Компоновщик. Класи що представляють функції, userstory, задачі будуть наслідуватися від одного інтерфейсу ITask, функції (Feature) та Userstory будуть складними об'єктами і міститимуть колекції об'єктів ITask, а задачі (Task) будуть представляли кінцеві об'єкти без дочірніх елементів.

Для розрахунку оціночної вартості робіт, в ITask інтефейс додаємо метод GetEstimatedPoints(). В класах-компоновщиках методи GetEstimatedPoints() реалізовуємо як обхід всіх дочірніх елементів та сумування результатів відповідей GetEstimatedPoints().

Таким чином, візуальні форми будуть працювати з колекцією елементів ITask і їм не потрібно буде знати конкретні типи дочірніх класів з якими вони працюють. В результаті логіка візуальних форм виходить достатньо простою і вона не буде містити бізнес-логіки розрахунку загальної оцінки по проєктам, а просто викликає метод GetEstimatedPoints() не замислюючись містить цей об'єкт дочірні об'єкти чи ні.

Переваги та недоліки:

- + Спрощує представлення деревоподібної структури.
- + Додає гнучкості в роботі з складними об'єктами та рекурсивними операціями.
- + Дозволяє додавати та видаляти об'єкти в ієрархії без впливу на клієнтський код.
- Потрібні додаткові зусилля для початкового впровадження.
- Вимагає гарно спроектованого загального інтерфейсу.

1.1.2. Шаблон «Flyweight»

Призначення: Шаблон використовується для зменшення кількості об'єктів в додатку шляхом поділу цих об'єктів між ділянками додатку. Flyweight являє собою поділюваний об'єкт.

Дуже важливою є концепція «внутрішнього» і «зовнішнього» станів [6]. Внутрішній стан відображає дані, характерні саме поділюваному об'єкту

(наприклад, код букви); зовнішній стан несе інформацію про його застосування в додатку (наприклад, рядок і стовпчик). Внутрішній стан зберігається в самому поділюваному об'єкті, зовнішній – в об'єктах додатку (контексту використання поділюваного об'єкта).

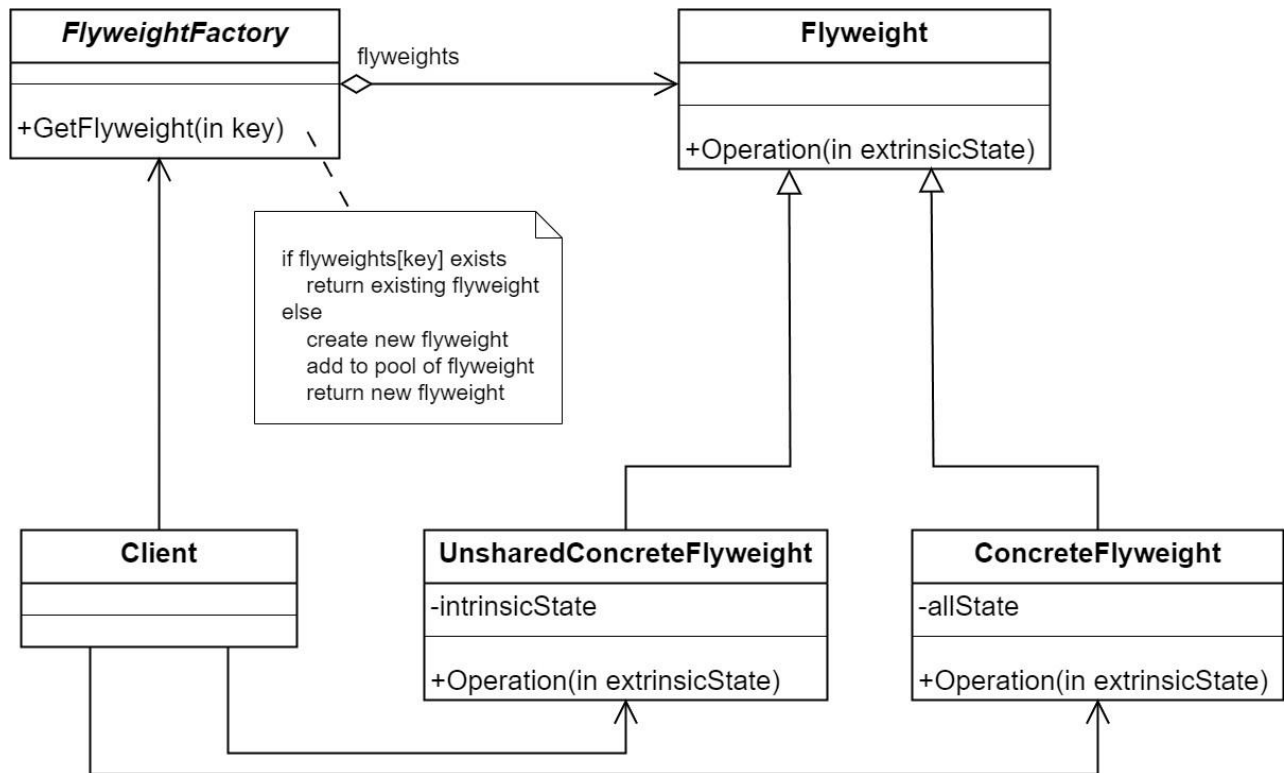


Рисунок 8.2. Структура патерну Flyweight (Легковаговик)

Хорошим прикладом є наступне зображення на рисунку 3.

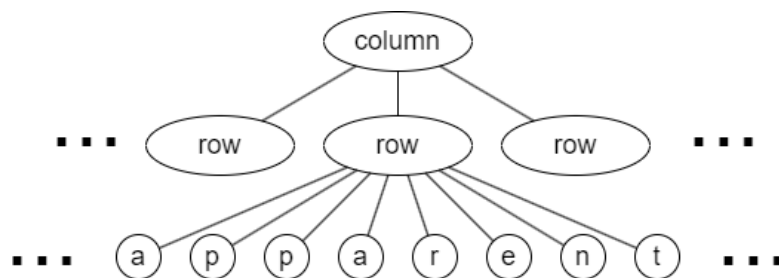


Рисунок 8.3. Приклад ситуації де варто використовувати Flyweight

Здається, ніби для кожної літери існує окремий об'єкт. Насправді фізично об'єкт всього один, існує лише безліч посилань на нього. (рисунку 8.4)

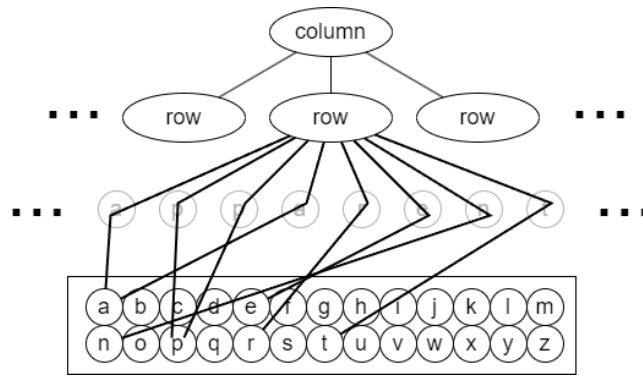


Рисунок 8.3. Використання патерну Flyweight

Даний шаблон дуже добре застосовувати у випадках, коли використовується безліч однакових об'єктів (наприклад, графічних примітивів).

Переваги та недоліки:

- + Заощаджує оперативну пам'ять.
- Витрачає процесорний час на пошук/обчислення контексту.
- Ускладнює код програми внаслідок введення безлічі додаткових класів.

1.1.3. Шаблон «Interpreter»

Призначення: Даний шаблон використовується для подання граматики і інтерпретатора для вибраної мови (наприклад, скриптової) [6]. Граматика мови представлена термінальними і нетермінальними символами, кожен з яких інтерпретується в контексті використання. Клієнт передає контекст і сформовану пропозицію в використовувану мову в термінах абстрактного синтаксичного дерева (деревоподібна структура, яка однозначно визначає ієрархію виклику підвиразів), кожен вираз інтерпретується окремо з використанням контексту. У разі наявності дочірніх виразів, батьківський вираз інтерпретує спочатку дочірні (рекурсивно), а потім обчислює результат власної операції.

Шаблон зручно використовувати в разі невеликої граматики (інакше розростеться кількість використовуваних класів) і відносно простого контексту (без взаємних залежностей і т.п.).

Даний шаблон визначає базовий каркас інтерпретатора, який за допомогою рекурсії повертає результат обчислення пропозиції на основі результатів окремих елементів.

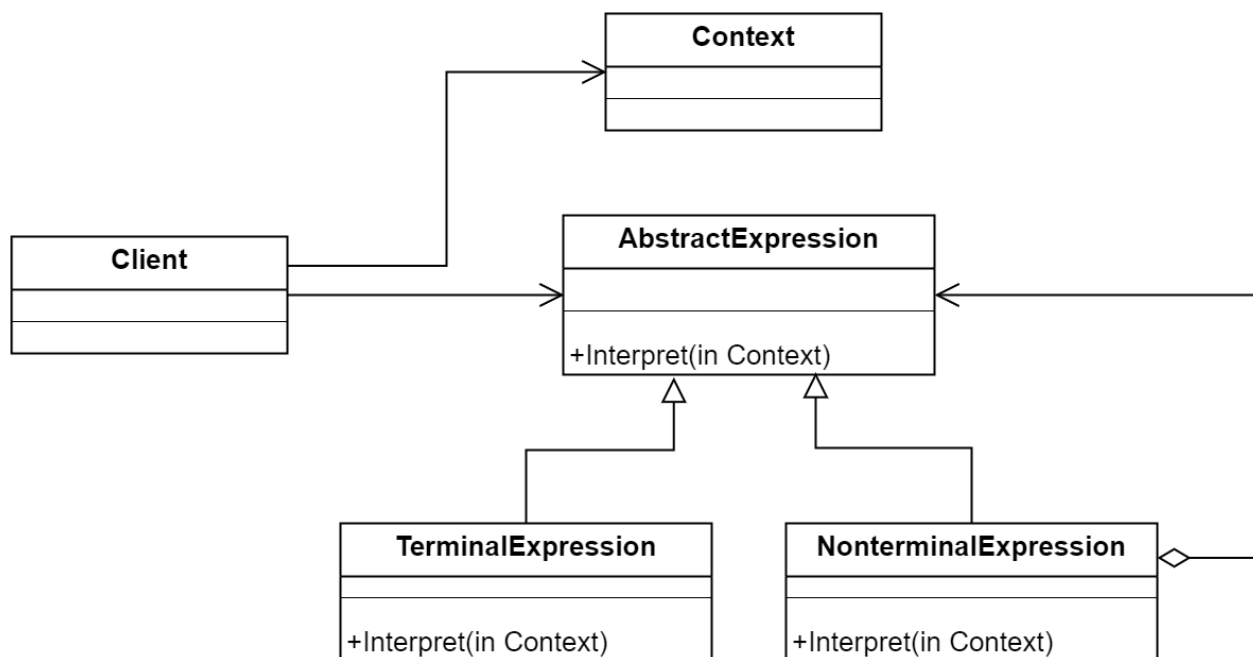


Рисунок 8.4. Структура патерна «Інтерпретатор»

При використанні даного шаблону дуже легко реалізовується і розширюється граматика, а також додаються нові способи інтерпретації виразів

Проблема: Стоїть задача пошуку рядків по зразку, як така, що часто зустрічається. Або ж загалом є якась задача, яка дуже часто змінюється.

Рішення: Може бути вирішена шляхом створення інтерпретатора, який визначає граматiku мови. «Клієнт» будує речення у вигляді абстрактного синтаксичного дерева, у вузлах якого знаходяться об'єкти класів «НетермінальнийВираз» і «ТермінальнийВираз» (рекурсивне), потім «Клієнт» ініціалізує контекст і виражає операцію Розібрати(Контекст). На кожному вузлі типу «НетермінальнийВираз» визначається операція Розібрати для кожного підвиразу. Для класу «ТермінальнийВираз» операція Розібрати визначає базу рекурсії. «АбстрактнийВираз» визначає абстрактну операцію Розібрати,

загальну для всіх вузлів в абстрактному синтаксичному дереві. «Контекст» містить інформацію, глобальну по відношенню до інтерпретатора.

Переваги та недоліки:

- + Граматику стає легко розширювати та змінювати, реалізації класів, що описують вузли абстрактного синтаксичного дерева схожі (легко кодуються).
- + Можна легко змінювати спосіб обчислення виразів.
- Супроводження граматики с великою кількістю правил є проблематичним.

1.1.4. Шаблон «Visitor»

Призначення: Шаблон відвідувач дозволяє вказувати операції над елементами без зміни структури конкретних елементів [6]. Таким чином вкрай зручно додавати нові операції, проте дуже важко додавати нові елементи в ієрархію (необхідно додавати відповідні методи для обробки їх відвідувань в кожного відвідувача).

Даний шаблон дозволяє групувати однотипні операції, що застосовуються над різнотипними об'єктами.

Проблема: Ви розробляєте онлайн-корзину інтернет магазину. Товари які представлені в магазині є різних типів, наприклад, електроніка, міцні напої, домашня хімія.

Логіка роботи з товарами в корзині є різна, наприклад, розрахунок вартості, формування замовлення.

Ми можемо всі ці методи зробити в товарах, але тоді ми ускладнюємо товари і змішуємо логіку (розрахунок вартості) з даними (товаром та його кількістю).

Якщо ми в подальшому необхідно буде додати ще логіку розрахунку вартості з врахуванням знижки, то потрібно буде додати ще цю логіку до товарів. А якщо буде ще сезонна знижка, то ми знову будемо добавляти нову логіку до класів товарів.

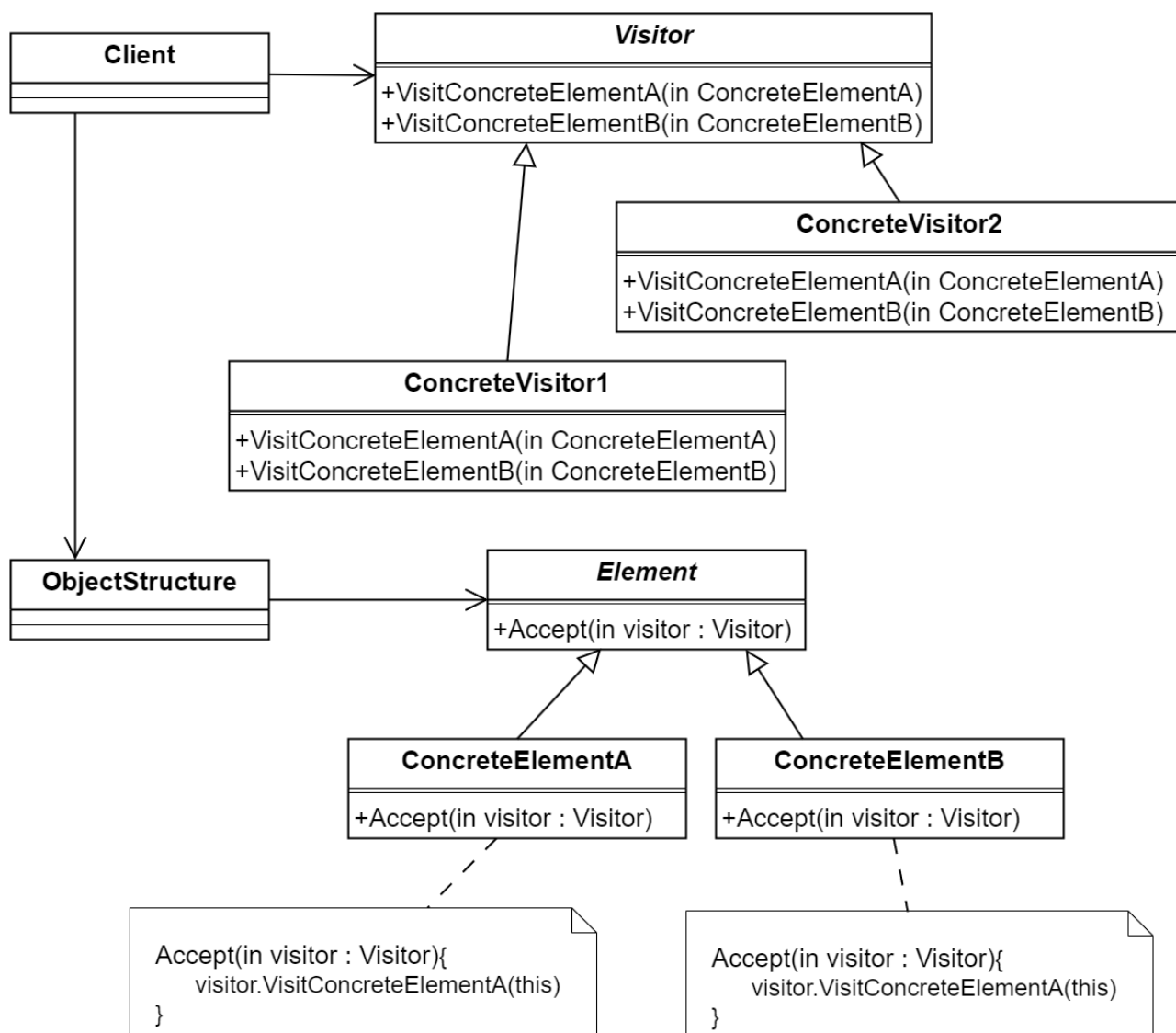


Рисунок 8.5. Структура патерна «Відвідувач»

Рішення: Основна ідея патерна «Відвідувач» це рознести логіку і дані в різні класи та ієрархії. Якщо так зробити для нашої онлайн-корзини, то в класі відвідувача ми маємо функції для обрахунку логіки для кожного типу, а об'єкти в корзині знають свій тип, отримують екземпляр відвідувача і в нього викликають метод відповідно до свого типу. В результаті ми робимо різні відвідувачі для розрахунку вартості: один для звичайних розрахунків, інший для розрахунку вартості зі знижкою, ще один для розрахунку сезонних знижок.

За рахунок того, що логіка відокремлена від наших товарів в корзині ми можемо реалізовувати за необхідності нові класи відвідувачі, а класи товарів та і корзини, в цілому, змінюватися не будуть.

Якщо розвивати далі, то можна зробити і клас відвідувача, який буде формувати замовлення з товарів в корзині в залежності від продавців і можливих варіантів доставки. Це дозволить формувати, наприклад, не одне замовлення, а два одна з самовивозом з магазину, а інше з доставкою Новою Поштою.

Приклад з життя: Прикладом може служити написання компілятора. Припустимо, існують різні об'єкти в синтаксисі мови програмування: виклики методів і умовні вирази. Компілятор перед генерацією коду повинен обійти всі вирази (і виклики методів, і умовні вирази) і перевірити безпеку типів, після чого згенерувати відповідний код. Відповідно буде два відвідувачі – для перевірки безпеки типів і для генерації коду. У кожного з них буде по 2 методи – для викликів методів і для умовних операцій. Таким чином при необхідності додавання нових кроків компіляції досить буде визначити нового «відвідувача» і викликати його у відповідний час.

Хід роботи

1. Створюю ArchiveVisitor та ArchiveOperation за патерном Visitor для відслідковування операцій з архівами

```
1 package com.beowulf.core.visitor;
2
3 import com.beowulf.core.interfaces.LogVisitor;
4 import com.beowulf.core.service.ArchivePersistenceService;
5
6 public class ArchiveVisitor implements LogVisitor {
7
8     private final ArchivePersistenceService service;
9
10    public ArchiveVisitor(ArchivePersistenceService service) {
11        this.service = service;
12    }
13
14    @Override
15    public void visit(ArchiveOperation context) {
16        service.persistOperation(context);
17    }
18 }
```

Рис. 1 ArchiveVisitor

```
1 package com.beowulf.core.visitor;
2
3 import com.beowulf.core.model.ArchivePart;
4 import com.beowulf.core.user.AppUser;
5
6 import java.util.ArrayList;
7 import java.util.List;
8
9 import java.util.UUID;
10
11 public class ArchiveOperation {
12
13     private AppUser user;
14
15     private String operation;
16     private String status;
17     private String archivePath;
18     private String targetPath;
19
20     private String format; // ZIP / TAR / RAR / ACE
21     private String compression; // GZIP / BZIP2 / LZMA / XZ / ZSTD
22     private String checksumType; // SHA256 / CRC32
23     private String checksumValue;
24     private long sizeBytes;
25     private long durationMs;
26
27     private UUID checksumId;
28     private UUID archiveId;
29
30     private List<ArchivePart> parts;
31
32     public AppUser getUser() {
33         return user;
34     }
35
36     public void setUser(AppUser user) {
37         this.user = user;
38     }
39
40     public String getOperation() {
41         return operation;
42     }
43
44     public void setOperation(String operation) {
45         this.operation = operation;
46     }
47
48     public String getStatus() {
49         return status;
50     }
51
52     public void setStatus(String status) {
53         this.status = status;
54     }
55
56     public String getArchivePath() {
57         return archivePath;
58     }
59
60     public void setArchivePath(String archivePath) {
61         this.archivePath = archivePath;
62     }
63
64     public String getTargetPath() {
65         return targetPath;
66     }
67
68     public void setTargetPath(String targetPath) {
69         this.targetPath = targetPath;
70     }
71 }
```

Рис. 2 ArchiveOperation частина 1

```

71 public class ArchiveOperation {
72     public String getFormat() {
73         return format;
74     }
75
76     public void setFormat(String format) {
77         this.format = format;
78     }
79
80     public String getCompression() {
81         return compression;
82     }
83
84     public void setCompression(String compression) {
85         this.compression = compression;
86     }
87
88     public String getChecksumType() {
89         return checksumType;
90     }
91
92     public void setChecksumType(String checksumType) {
93         this.checksumType = checksumType;
94     }
95
96     public String getChecksumValue() {
97         return checksumValue;
98     }
99
100     public void setChecksumValue(String checksumValue) {
101         this.checksumValue = checksumValue;
102     }
103
104     public long getSerializedBytes() {
105         return serializedBytes;
106     }
107
108     public void setSerializedBytes(long serializedBytes) {
109         this.serializedBytes = serializedBytes;
110     }
111
112     public long getDurationMs() {
113         return durationMs;
114     }
115
116     public void setDurationMs(long durationMs) {
117         this.durationMs = durationMs;
118     }
119
120     public UUID getChecksumId() {
121         return checksumId;
122     }
123
124     public void setChecksumId(UUID checksumId) {
125         this.checksumId = checksumId;
126     }
127
128     public UUID getArchiveId() {
129         return archiveId;
130     }
131
132     public void setArchiveId(UUID archiveId) {
133         this.archiveId = archiveId;
134     }
135
136     public List<ArchivePart> getParts() {
137         return parts;
138     }
139
140     public void setParts(List<ArchivePart> parts) {
141         this.parts = parts;
142     }
143
144     public void addPart(ArchivePart part) {
145         if (part == null) {
146             return;
147         }
148         if (this.parts == null) {
149             this.parts = new ArrayList<>();
150         }
151         this.parts.add(part);
152     }
153
154 }

```

Рис. 3 ArchiveOperation часть 2

2. Розроблюю діаграму класів

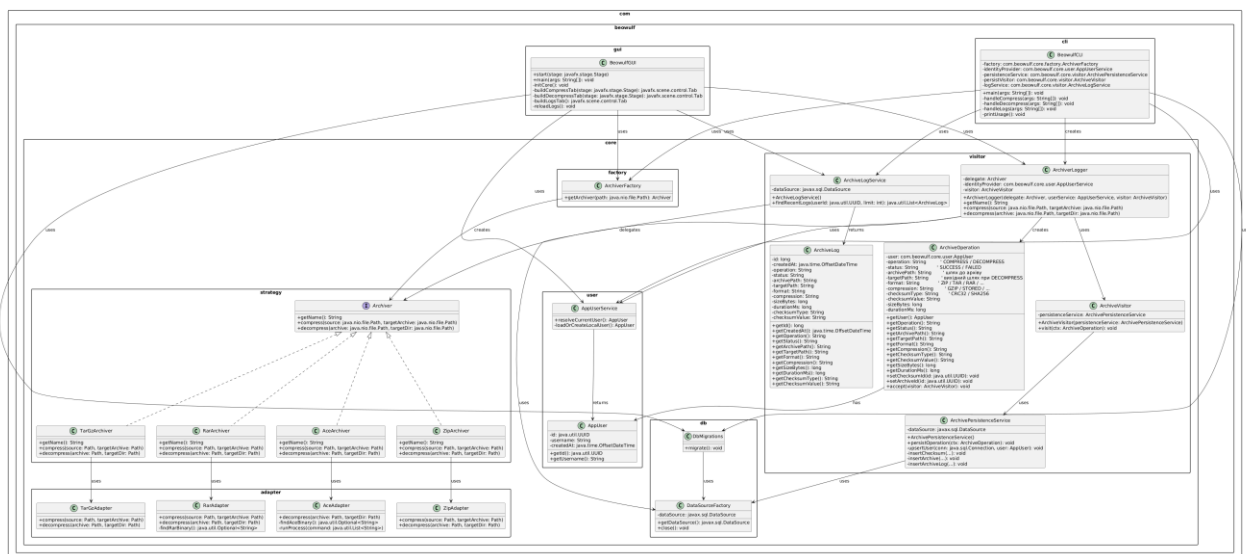


Рис. 4 Діаграма класів

Висновок

У ході цієї лабораторної роботи я закріпив теоретичні знання про шаблони «Composite», «Flyweight», «Interpreter» та «Visitor» і розглянув їхні можливості для підвищення гнучкості та масштабованості програмних систем. Я проаналізував, як «Composite» спрощує роботу з деревоподібними структурами, «Flyweight» дозволяє оптимізувати використання пам'яті за рахунок спільного внутрішнього стану, «Interpreter» надає спосіб формалізувати граматику і будувати інтерпретатор для вибраної мови, а «Visitor» дає можливість додавати нові операції без зміни класів елементів. Отримані результати показали, що використання цих патернів робить код більш структурованим, зручним у підтримці та здатним адаптуватися до розширення функціональності в майбутньому.

Контрольні запитання

1. Яке призначення шаблону «Композит»?

(Дати можливість працювати з окремими об'єктами і групами об'єктів однаково, організувавши їх у деревоподібну структуру)

2. Нарисуйте структуру шаблону «Композит».

(Складається з базового компонента, листків та контейнерів, де контейнер містить набір підкомпонентів і реалізує ті самі операції, що й листок)

3. Які класи входять в шаблон «Композит», та яка між ними взаємодія?

- ✧ Component — спільний інтерфейс.
- ✧ Leaf — елементарний об'єкт без підкомпонентів.
- ✧ Composite — містить колекцію компонентів і делегує їм операції.

(Взаємодія: клієнт працює з Component, а Composite сам викликає методи своїх дітей)

4. Яке призначення шаблону «Легковаговик» (Flyweight)?

(Зменшити використання пам'яті, розділяючи спільний внутрішній стан між багатьма легкими об'єктами)

5. Нарисуйте структуру шаблону «Легковаговик».

(Складається з Flyweight (спільний стан), ConcreteFlyweight (конкретна реалізація) і FlyweightFactory (керує створенням та кешуванням спільних об'єктів))

6. Які класи входять в шаблон «Легковаговик», та яка між ними взаємодія?

- ✧ Flyweight — інтерфейс легковаговика.
- ✧ ConcreteFlyweight — реалізація спільного стану.
- ✧ FlyweightFactory — повертає вже створені Flyweight або створює нові.

(Взаємодія: клієнт працює з легковаговиками через фабрику, яка забезпечує їх повторне використання)

7. Яке призначення шаблону «Інтерпретатор»?

(Описувати граматику мови та інтерпретувати вирази цієї мови за допомогою класів, що представляють правила граматики)

8. Яке призначення шаблону «Відвідувач»?

(Винести операції над елементами складної структури в окремий клас, дозволивши додавати нові операції без зміни класів елементів)

9. Нарисуйте структуру шаблону «Відвідувач».

(Складається з Visitor (набір операцій), ConcreteVisitor, Element (інтерфейс елементів) і ConcreteElement (елементи, що приймають відвідувача через метод accept))

10. Які класи входять в шаблон «Відвідувач», та яка між ними взаємодія?

✧ Visitor — описує операції для кожного типу елементів.

✧ ConcreteVisitor — конкретні реалізації операцій.

✧ Element — має метод accept.

✧ ConcreteElement — викликає відповідний метод у Visitor.

(Взаємодія: клієнт передає visitor елементам через accept, а visitor виконує потрібну операцію залежно від типу елемента)